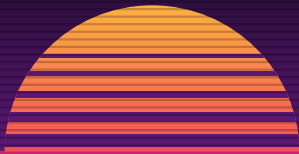


From Audio to Beatmap

how the backend turns any song into a five-difficulty osu! map



AUDIO > STFT > FLUX > TEMPO > OBJECTS > .OSZ

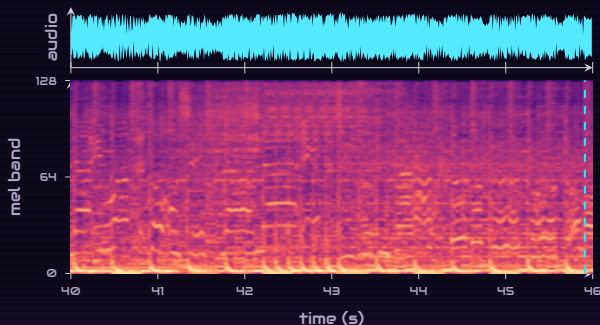
One song in, five maps out

A beatmap is a list of objects, and each one needs three answers:



OUT One command, `python src/main.py song.mp3`, gives five difficulties from Easy to Expert that the real game imports.

Seeing the song: the mel spectrogram



SHORT-TIME FOURIER TRANSFORM

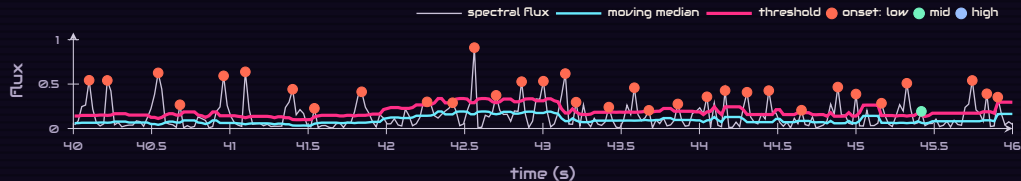
$$X[m, k] = \sum_n x[n + mH] w[n] e^{-j2\pi kn/N}$$

mono, 22050 Hz, peak-normalised

- > 92.9 ms Hann window, 23.2 ms hop: 43.1 spectra per second
- > pooled into 128 mel bands, fine in the bass and coarse up high, the way hearing works

OUT The vertical streaks are broadband **attacks**. Those are the note starts we want to find.

Finding every note start



SPECTRAL FLUX

$$SF[m] = \|\max(0, P_m - P_{m-1})\|$$

energy that appears; fading notes are ignored

ADAPTIVE THRESHOLD

$$T[m] = \delta + \lambda \cdot \text{median}_{\pm 0.5s} SF$$

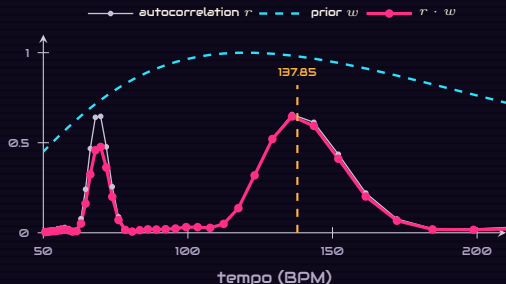
tracks quiet verses and loud drops alike

PEAK PICKING

local maxima above T , at least 50 ms apart, each tagged with its strongest frequency band

OUT 862 onsets in 219.1s, each one carrying its strength and band (low / mid / high).

How fast is the music?



AUTOCORRELATION (WIENER-KHINCHIN)

$$r = \mathcal{F}^{-1}\{|\mathcal{F}\{e\}|^2\}$$

a beat is a repeating ripple in the flux; r peaks at its period

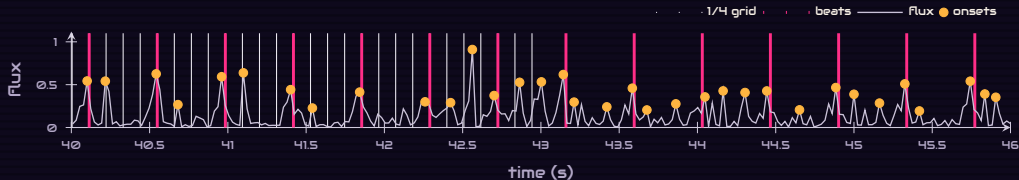
PRIOR + OCTAVE CHECK

a log-normal prior around 120 BPM, then: which pulse train really lands on the hits?

CANDIDATE	BPM	SCORE
half	68.9	1.02
Found	137.8	1.35
double	275.7	0.52

OUT Autocorrelation finds the period; the prior and octave check keep it off half or double tempo. Result: **137.85 BPM**.

Locking to the beat grid



PHASE: WHERE IS BEAT ONE?

$$\varphi^* = \arg \max_{\varphi} \frac{1}{N} \sum_n e(\varphi + nP)$$

slide a pulse train across one beat; keep the offset that collects the most energy

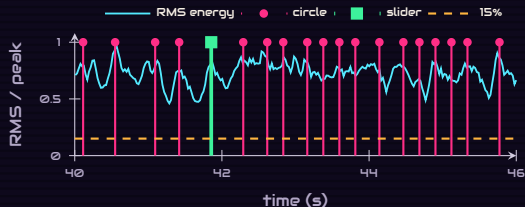
SNAP: EACH ONSET TO ITS NEAREST SLOT

$$\hat{t} = \varphi^* + \text{round}\left(\frac{t - \varphi^*}{P/d}\right) \frac{P}{d}$$

removes detector jitter; two onsets in one slot keep the stronger (77 merged)

OUT One constant grid (offset 70 ms, a beat every 435.3 ms) is what makes a map feel snapped to the music.

Circle, slider or spinner?



Sustain ratio s : the share of frames after a hit that stay above 15% of peak. Near 1, the note rings on; near 0, it decayed.

SPINNER

gap of 3+ beats and the sound is held

STREAM SLIDER

a run of fast onsets under half a beat apart

HELD SLIDER

gap of 1-2 beats and the sound is held

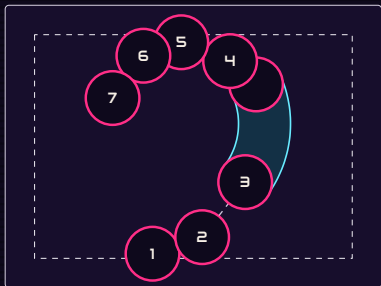
CIRCLE

anything else: a hit that decays

checked top to bottom; the first rule that fits wins

OUT The shape of the energy after a hit decides the object: a tap decays, a hold rings on.

Placing objects like a mapper would



Hard: 7 objects in play order, true scale

DISTANCE SNAP

$$d = D \cdot \sigma \cdot \Delta b$$

distance grows linearly with the gap Δb in beats, in circle diameters D ; σ runs from 0.85 (Easy) to 2.10 (Expert)

A VIRTUAL CURSOR WALKS THE MAP

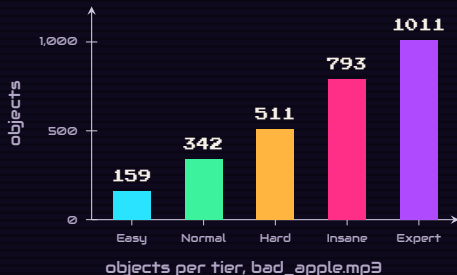
- > flow: small turns over short gaps so streams arc; sharp turns only on jumps
- > never bury: at least $0.75 D$ from the previous object
- > stay off slider bodies: clear the whole drawn curve, not just its ends

OUT Mapping conventions become geometry: how far apart two objects are tells the player the rhythm.

One analysis, five difficulties

KNOB	Easy	Hard	Expert
threshold λ / δ	2.2 / .08	1.5 / .05	1.1 / .03
snap grid	1/1	1/4	1/4
min spacing (beats)	2.0	0.5	0.25
distance σ	0.85	1.3	2.10
max turn	40°	75°	130°
AR / OD / CS	4 / 3 / 3.2	7.5 / 6 / 4	9.3 / 8.5 / 4.2

The **DSP knobs** set how many onsets exist. The **musical knobs** set how many survive and how they move.



OUT The audio is analysed once and re-read with five presets, so one song becomes a full beatmap set.

A File the real game accepts

OSU FILE FORMAT V14

```
[Difficulty]
HPDrainRate:5
CircleSize:4
OverallDifficulty:6
ApproachRate:7.5
SliderMultiplier:1.4
SliderTickRate:1
```

```
[TimingPoints]
70,435.260746,4,1,0,70,1,0
```

```
[HitObjects]
69,192,40114,1,0,0:0:0:0:
326,240,41855,2,0,P|352:175|340:105,1,140.00,0|0,0:0:0:0:0:0:
256,192,89189,8,0,90930,0:0:0:0:
```

Five .osu Files + the audio, zipped into one .osz; an analysis.json keeps the onsets, grid and spectrogram for the app to show

NO GROUND TRUTH, SO TEST THREE WAYS

- > synthetic signals: click tracks at a known BPM, held tones, fast bursts, each with an exact expected answer
- > listening: a click on every onset and a metronome on every beat; drift is obvious by ear
- > play-testing in osu!lazer; every problem found became an automated test

pytest: 256 backend tests

OUT osu!lazer rejects malformed Files, so a clean import doubles as the end-to-end test.